



TECHNICAL PROFESSIONAL

ICT - COMPUTER PROGRAMMING, JAVA

Grade 12

**Three-Term Budget Of Work (BOW)
for Learning Competencies**

TECHNICAL PROFESSIONAL

ICT - ICT SUPPORT AND COMPUTER PROGRAMMING TECHNOLOGIES

Grade 12

Three-Term Budget Of Work For Learning Competencies

Please note that each TechPro elective is completed within one term if delivered at Grade 12.

Term Plan	
Elective	Computer Programming, Java
Performance Standard	The learners perform procedures in setting up the Integrated Development Environment (IDE) and in creating algorithms, flowcharts, and pseudocode to plan and visualize their code. The learners create a well-structured program using Java Operators, input and output classes, flow control structures, and arrays. The learners create a well-structured and efficient program using Java flow control structures, Java arrays, and Java Object-Oriented Programming (OOP) language.
Content Standard	• The learners demonstrate an understanding of Java programming, including basic structure, syntax, data types, variables, and comments. They also develop skills in creating algorithms, flowcharts, and pseudocode to plan and visualize their code before implementation. • The learners demonstrate an understanding of Java Operators, Input/Output (I/O) classes, flow control structures, and arrays to solve real-world problems. • The learners demonstrate an understanding of object-oriented Java programming principles, procedures to manage errors with exception handling, and the Java Collections Framework for efficient data management. • The learners showcase their grasp of Java's multithreading principles, graphical user

interface (GUI) development, and efficient component structuring to build interactive and responsive applications. They incorporate market analysis to ensure their solutions meet industry demands and customer needs.

Week	Learning Competency	Suggested Activities
1	- Discuss the fundamental concepts of Java. - Perform the steps in setting up the development environment. - Discuss Java basic structures and syntax. - Develop algorithms for designing a program or system.	Java Concepts Quick Map - Learners identify the key Java concepts such as program structure, syntax, data types, variables, and comments. - Learner will make a simple concept map showing how these elements appear in a basic Java program. Java Program Walk-Through - Learners explain the fundamental parts of a simple Java program. - Through a given a sample code, learners label and briefly describe the purpose of each component (class, main method, variables, comments). Java Tools Setup Check - Learners prepare their computer for Java programming. - Learners install the Java Development Kit (JDK), open the IDE, and check if Java

Week	Learning Competency	Suggested Activities
		runs correctly by viewing the version or welcome screen. • First Project Test Run ○ Learners confirm that the development environment works properly. ○ Learners create a new Java project in the IDE and run a simple sample program to see if it executes without errors. • Parts of a Java Program ○ Learners identify the basic parts of a Java program. ○ Learners are given a simple Java code and label the class name, main method, variables, and comments, then briefly explain what each part does. • Syntax Spot-the-Error ○ Learners recognize correct and incorrect Java syntax. ○ Learners review short Java code samples and point out missing semicolons, brackets, or wrong keywords, then correct them. • Everyday Steps to Algorithm ○ Learners learn how an algorithm works

Week	Learning Competency	Suggested Activities
		using a familiar activity. ○ Learners choose a simple daily task (e.g., logging in to a computer) and write the step-by-step instructions in correct order as an algorithm. ● Problem-to-Algorithm Challenge ○ Learners create an algorithm to solve a simple programming problem. ○ Given a basic problem (e.g., display the sum of two numbers), learners list the steps needed to solve it using clear and logical sequence.
2	● Develop flowcharts for designing a program or system. ● Develop pseudocodes for designing a program or system.	● From Steps to Flowchart ○ Learners convert simple steps into a flowchart. ○ Learners take a given step-by-step solution (algorithm) and draw the correct flowchart using basic symbols (start, process, decision, end). ● Flowchart a Simple Problem ○ Learners design a flowchart for a basic programming problem. ○ Given a simple task (e.g., display the larger of two numbers), learners draw a

Week	Learning Competency	Suggested Activities
		flowchart showing the logical flow from start to end. • Algorithm to Pseudocode ○ Learners translate a simple solution into pseudocode. ○ Given a basic problem (e.g., display the sum of two numbers), learners write the steps in plain-English pseudocode using correct sequence and keywords. • Pseudocode Fix-It Task ○ Learners improve a given pseudocode to make it logical and complete. ○ Learners review a short pseudocode with missing or incorrect steps, then revise it so the logic is clear and ready for coding.
3	• Apply operators in writing a Java program. • Apply Java I/O classes to read and write data files in Java programs.	
4	• Apply Java Flow Control Structures in writing a Java program. • Apply Java Arrays in writing a Java program.	
5	• Apply sorting algorithms in a Java program that organizes customer transaction data stored in an array.	
*	• Discuss Object-Oriented Programming (OOP) Language in Java. • Apply OOP principles in a Java program.	

Week	Learning Competency	Suggested Activities
	• Apply exception handling in the Java program. • Apply Java Collections Framework in writing Java programs. • Apply the java.io package and java file operations in a Java program. • Apply reading and writing classes in writing a Java program.	
*	• Apply threads in creating a Java program using both the Thread class and Runnable interface. • Create a Java program that effectively integrates CountdownLatch, CyclicBarrier, Semaphore. • Create simple GUI applications by using JFrame, JLabel, JButton, and JTextField while handling events with ActionListeners. • Create user interfaces in Java using layout managers such as FlowLayout, BorderLayout, and GridLayout, while effectively nesting components within multiple JPanels. • Create interactive Java applications by leveraging advanced UI components and handling complex user interactions to enhance user experience and interface responsiveness. • Develop business plans for a system proposal. • Create designs of a GUI Java application showcasing creativity to solve real-world problems.	
Suggested Performance Tasks		• Individual Learning Activity : My Java Workspace Setup ○ Learners independently set up the Java IDE and prepare their workspace for coding. ○ Learners install the Java Development Kit (JDK), open and configure the IDE, then create and save a new Java project to confirm the setup is ready for coding. • Group Activity : Code Planning Team Lab ○ Learners work in groups to plan a simple program before coding. ○ In small groups, learners select a simple problem, agree on the solution steps, and create an algorithm, flowchart, and pseudocode to clearly visualize the program logic.

* Teachers may assign learning competencies per week depending on the needs of the learners.